

XM · CLASE 6



Soluciones de Analítica con Databricks

Clase 6: Modelado de plata: de reporte a tabla analítica

Docente: Manuela Larrea Gómez

Agenda de la clase 6



Hoy la teoría vive en el notebook: se recorre, se ejecuta y se discute sobre tus datos

01

10 MIN

Encuadre y sector

Qué construimos (demanda_diaria, dim_ciiu) · CIIU Rev. 4 A.C. y por qué solo aplica al mercado no regulado · hito 3

02

50 MIN

Notebook guiado, con la docente

Largo → ancho · SQL vs. PySpark · MERGE idempotente · time travel · dimensión CIIU · liquid clustering, OPTIMIZE y Photon · contrato

03

50 MIN

Lab 6: el mismo notebook, solo

normalizar_texto y ALIAS_CIIU en src · las tres preguntas · verificar() · commit y PR

04

10 MIN

Cierre

Hito 3 cerrado · tag s06 · así sería en Azure · próxima clase: features y oro

El notebook de hoy: siete secciones, cada una con teoría y código

clases/c06-silver/notebooks/lab06_silver_modelado · widget usuario · requiere demanda_validada (lab 5)

#	Sección	Teoría que se explica	Lo que se ejecuta / se ve
1	Largo → ancho	Grano, contrato de plata, snake_case	Tabla del contrato de demanda_diaria
2	El pivote	Cuándo SQL y cuándo PySpark	Dos versiones, mismo resultado: 72.704 serie-días
3	MERGE idempotente	overwrite vs. append vs. MERGE; la condición que da idempotencia	DESCRIBE HISTORY: 72.704 insertadas · 0 · 3 actualizadas
4	Time travel	Versiones, VERSION AS OF, RESTORE, retención	El mismo serie-día en v2 y v3
5	Dimensión CIIU	Hechos y dimensiones, texto normalizado, anti-join, alias	1 texto sin sección → alias → 22 secciones; ciuu_seccion en los hechos
6	Rendimiento	Particiones vs. liquid clustering, OPTIMIZE, Photon y UDFs	DESCRIBE DETAIL antes y después
7	Contrato	Comentarios y tags en plata	Catalog Explorer → demanda_diaria

Dos TODO en src/xm_demanda/silver/transform.py: normalizar_texto() y una entrada en ALIAS_CIIU. Todo lo demás viene hecho.

Sector: la CIIU y por qué solo aplica al mercado no regulado

Clasificación Industrial Internacional Uniforme, Rev. 4 adaptada para Colombia (DANE)

21

secciones (A–U) en la CIIU Rev. 4 A.C.; en nuestros datos aparecen las 21

29 series

regulado: 69 % de la energía, todas SIN CLASIFICAR

326 series

no regulado: 31 % de la energía, con sección CIIU

1

texto de la fuente que no coincide con el nombre oficial (lo encuentra el anti-join)



Regulado

Hogares y pequeños comercios con tarifa fijada por la CREG. Se comercializan por agente y mercado, no por actividad económica: llegan como SIN CLASIFICAR (sección X en nuestra dimensión).



No regulado

Empresas que negocian su tarifa con el comercializador. Se reportan por sección CIIU: por eso el modelo puede usar el sector económico como variable, solo en este tercio de la energía.

Fuente: dane.gov.co → Nomenclaturas y clasificaciones → CIIU Rev. 4 A.C. · creg.gov.co (usuarios regulados y no regulados)

Tres capacidades de Delta que hoy se ven en una sola tabla

Cada una responde a una pregunta que en XM alguien va a hacer

Capacidad	Pregunta que responde	Cómo	Cuándo NO
MERGE	¿Cómo aplico una republicación sin duplicar ni borrar el resto?	MERGE ... WHEN MATCHED AND s.pub > t.pub THEN UPDATE; WHEN NOT MATCHED THEN INSERT	En bronce (append) · sin llave de negocio clara
Time travel	¿Qué valor tenía antes? ¿Con qué datos se entrenó el modelo del día X?	VERSION AS OF · TIMESTAMP AS OF · RESTORE; el entrenamiento guarda la versión	Reproducibilidad a años: VACUUM borra versiones; se copia o se sube la retención
Liquid clustering + OPTIMIZE	¿Cómo acelero los filtros por fecha y mercado sin adivinar particiones?	ALTER TABLE ... CLUSTER BY (fecha, tipo_mercado); OPTIMIZE; predictive optimization en serverless	Tablas pequeñas (< cientos de MB): no se nota

Photon acelera SQL y DataFrames, no UDFs de Python: por eso normalizar_texto solo se usa para construir la dimensión (22 filas).

Fuente: docs.databricks.com/aws/en/delta/merge · docs.databricks.com/aws/en/delta/history · docs.databricks.com/aws/en/delta/clustering

Free Edition vs. Azure Databricks: qué simulamos hoy

Todo lo de hoy funciona igual; cambia la escala y quién lo ejecuta

Elemento	Hoy en Free Edition	En XM sobre Azure
MERGE de plata	Lo ejecutas tú desde el notebook, en tu esquema	Tarea del job diario (clase 12) o pipeline declarativo con AUTO CDC
Time travel	Versiones de tu tabla; retención por defecto 7 días	Retención ampliada en tablas de entrenamiento; versión registrada en MLflow (clase 9)
OPTIMIZE	Manual, para verlo	Predictive optimization automático; OPTIMIZE programado en tablas críticas
Datos de referencia (CIU)	CSV en el repo (data/reference)	Tabla de referencia gobernada en prod, con owner y versionado por MERGE
Photon	Serverless: siempre activo	Igual; se ve en el plan de ejecución (Spark UI) qué operadores lo usan

Lab 6: el mismo notebook, solo, con dos TODO en src

Después del recorrido con la docente, cada uno lo ejecuta en su esquema de principio a fin

1

Secciones 1–4

Ejecutar y leer. Responder la pregunta del MERGE sin condición

2

Sección 5: `normalizar_texto()`

En `transform.py`; la celda de prueba la valida

3

Sección 5: el anti-join

Un texto sin sección → `ALIAS_CIIU` → 22 secciones, 0 sin match

4

Secciones 6–7

`CLUSTER BY + OPTIMIZE`; comentarios y tags

5

`verificar()` · `commit` · `PR`

`feat(c06)`: plata en formato ancho con `MERGE` y `dim_ciiu`

Criterios de aceptación (hito 3)

- `demanda_diaria`: 72.704 filas, llave única, `snake_case`, `ciiu_seccion` sin nulos
- Historial con un `MERGE` de 3 actualizadas y uno que no tocó nada
- `CLUSTER BY` (`fecha`, `tipo_mercado`); comentario de tabla
- `dim_ciiu`: 22 secciones, todas con `nombre_fuente`
- `test_silver.py` en verde; `verificar()` en OK; `PR` en verde

Comparar con s06

```
github.com/<docente>/Databricks-to-XM/compare/s06...<tú>:Databricks-to-XM:main · o git diff s06 --src/xm_demanda/silver clases/c06-silver/notebooks
```

Así sería en Azure Databricks: plata como tabla de producto

Hoy	En XM
MERGE lanzado desde el notebook	Tarea silver_merge del job diario, después del pipeline de calidad; alerta si actualiza más filas de lo esperado
Republicación simulada (3 filas)	Llega sola por bronce → validada → MERGE; el historial de plata registra cuántas filas cambió cada publicación de XM
Versión de la tabla anotada a mano	El entrenamiento registra la versión Delta en MLflow; auditoría reproduce cualquier pronóstico
dim_ciiu desde un CSV del repo	Tabla de referencia gobernada (owner: grp_analitica) con MERGE cuando el DANE actualiza la clasificación
OPTIMIZE a mano	Predictive optimization; liquid clustering definido en el bundle (resources/) para cada tabla

Lo que no cambia: el contrato de demanda_diaria, la llave, la condición del MERGE, la dimensión y sus alias en src.

Resumen de la clase 6 y preparación para la clase 7

Logros de hoy

✓ Largo → ancho

Plata habla el idioma de quien la consume: una fila por serie-día

✓ MERGE idempotente

La condición por `_publication_date` es lo que lo hace idempotente

✓ Time travel

Qué versión vio el modelo el día X: `VERSION AS OF`

✓ Dimensión CIU

Texto normalizado + anti-join + alias en src

✓ Hito 3 cerrado

`demanda_diaria` y `dim_ciu`, documentadas y clusterizadas

Próxima clase

➔ Sumar fuentes y construir features

Festivos, calendario, lags y ventanas móviles

➔ Oro: `features_demanda_diaria`

Una fila por serie-día con lo que el modelo necesita

➔ Feature engineering reproducible

Funciones en src, probadas; tabla con contrato

➔ Traer a la clase 7

Lab 6 terminado: oro se construye sobre `demanda_diaria`



Gracias

Nos vemos en la clase 7: Sumar fuentes y construir features (Gold)

DOCENTE

Manuela Larrea Gómez